LoanOps AI

MyData-driven Financial Decision Agent

기존 LoanOps 를 개인 금융 Digital Twin, 금융상품 매칭, 대출·주택구매 시뮬레이션, 스트레스 테스트, AI Agent 로 확장하는 구체화 계획

PROJECT THESIS

신뢰 가능한 금융상품 데이터(LoanOps)와 개인 마이데이터를 결합해, 사용자가 큰 금융 결정을 내리기 전 “가능한가?”뿐 아니라 “결정 후 내 재무상태가 어떻게 바뀌는가?”까지 계산하고 설명하는 의사결정 지원 Agent.

Planning Document | 2026.09

※ 포트폴리오/프로토타입 기획 문서. 실제 금융상담, 대출승인 또는 투자자문을 대체하지 않음.

1. Executive Summary

LoanOps v1 을 버리는 것이 아니라, 신뢰 가능한 금융 데이터 백본으로 승격한다.

LoanOps v1 의 핵심 가치는 여러 금융 데이터 소스를 단순히 모으는 것이 아니라, 원본 보존, 정규화, 데이터 품질검증, 오류 격리, 변경감지, 검증 데이터 서빙까지 "금융 데이터가 믿을 만한 상태로 사용자에게 도달하는 과정"을 구현했다는 점이다.

LoanOps AI 는 이 데이터 백본 위에 금융결제원 Open Banking 개발/테스트 환경, 공공 MyData 계약에 맞춘 Adapter, 그리고 개인 금융데이터 분석·시뮬레이션을 추가한다. Open Banking 쪽은 OAuth 2.0 과 실제 API 계약을 직접 연동하고, 공공 MyData 는 개인 포트폴리오에서 바로 운영기관 승인을 받기 어렵다는 현실을 반영해 공식 묶음정보/API 구조와 동일한 Mock Provider 로 구현한다. 사용자의 계좌·거래·카드·대출·소득 근거를 하나의 Personal Financial Twin 으로 정규화하고, 기존 LoanOps 상품 데이터와 결합해 새로운 대출이나 주택구매가 현금흐름·유동성·부채부담에 미치는 영향을 계산한다. AI Agent 는 숫자를 직접 만들어내지 않고 검증된 Tool 을 호출하고 결과를 설명한다. 또한 LoanOps Serving 상품을 단순 조회용으로 두지 않고 Product Matching Engine 의 후보군으로 사용한다. 사용자의 목표 대출규모, 감당 가능한 월 상환액, 기간·상환방식 선호와 Financial Twin 의 부채여력을 먼저 계산한 뒤, 조건이 맞는 대출상품만 검토 후보로 추리고 각 후보를 동일한 시뮬레이션·스트레스 테스트에 넣어 비교한다.

한 문장 정의

마이데이터 기반으로 개인의 현재 금융상태를 복제한 Financial Twin 을 만들고, 대출·주택구매 등의 금융 의사결정을 What-if Simulation 과 Stress Test 로 검증해 주는 AI Financial Decision Agent.

MVP 의 핵심 4 요소

구성요소	역할	LoanOps 와의 연결
Personal Financial Twin	자산·부채·소득·지출·현금흐름을 하나의 재무상태로 요약	개인 데이터 계층 추가
Home / Loan Simulator	대출 및 주택구매 전후 재무상태를 계산	기존 Serving Loan Product 활용
Stress Test	금리상승·소득감소 등 충격에 대한 내구성 계산	상품금리·대출조건을 시나리오에 사용
Product Matching Engine	Financial Twin 과 목표 시나리오를 기준으로 대출상품 후보를 필터링·비교하고 각 후보를 재시뮬레이션	ServingLoanProduct/Term/RateObservation 을 실제 사용자 의사결정에 연결
AI Agent	자연어 요청을 시나리오로 변환하고 Tool 을 호출해 설명	분석값/출처/가정을 Audit Trail 로 연결

왜 지금 이 프로젝트인가

- 금융 마이데이터는 단순 통합조회에서 개인의 의사결정과 실행을 지원하는 방향으로 확장되고 있다.
- 2026 년 금융위원회는 My AI Agent 도입과 데이터 활용 확대를 포함한 마이데이터 발전방안을 논의했고, 마이데이터 기반 금리인하요구 서비스도 혁신금융서비스로 지정했다.
- 따라서 "마이데이터 → 데이터분석 → 금융 의사결정 → Agent" 흐름 자체가 금융 AX 의 현재 방향성과 맞닿아 있다.
- 포트폴리오 측면에서는 데이터엔지니어링, 금융도메인 분석, 백엔드, 시뮬레이션, AI Agent 를 하나의 서비스로 연결할 수 있다.

2. Motivation & Problem Definition

"금리와 상품을 보여주는 것"에서 "내 결정을 검증해 주는 것"으로.

2.1 기존 LoanOps 가 해결한 문제

금융상품 데이터는 소스별 필드 구조, 기관명, 금리표현, 갱신주기, 누락·오류 패턴이 다르다. 기존 LoanOps 는 이를 공통 모델로 정규화하고 품질검증을 거쳐 Serving 데이터로 승격함으로써 "신뢰 가능한 금융상품 데이터 레이어"를 만든다.

현재 영역	기존 LoanOps 의 역할
Ingestion	FSS, 서울시 등 외부 금융 데이터 수집
Raw / Provenance	원본 응답과 수집 시점, 해시 등 추적정보 보존
Normalization	기관·상품·금리·기간 등 공통 도메인 모델로 변환
Quality	유효성 규칙, 격리(Quarantine), 검증 실패 관리
Publication	검증된 데이터만 Serving 영역으로 승격
Change Detection	상품 조건이나 금리 변화 감지

2.2 그러나 사용자의 실제 질문은 상품조회에서 끝나지 않는다

사용자가 실제로 궁금한 것은 "어떤 대출이 있나?"보다 "내가 이 대출을 받아도 되는가?", "7 억원짜리 집을 지금 사면 생활이 버틸 수 있는가?", "금리가 2%p 오르면 얼마나 위험해지는가?"에 가깝다.

> **Problem Gap**
>
> LoanOps v1 은 금융상품 데이터의 신뢰성 문제를 해결했지만, 개인의 금융상태와 상품 데이터를 연결해 "결정의 결과"를 계산하지는 않는다. LoanOps AI 는 이 공백을 메운다.

2.3 해결하고자 하는 핵심 문제

1. 개인 금융정보가 계좌·카드·대출·투자에 분산되어 있어 전체 재무상태를 한눈에 판단하기 어렵다.
2. 대출한도나 월 상환액을 알아도, 그 결정이 이후 현금흐름과 비상자금에 어떤 영향을 미치는지 연결해서 보기 어렵다.
3. 금리상승·소득감소와 같은 미래 위험을 실제 개인 데이터 기준으로 스트레스 테스트하기 어렵다.
4. 생성형 AI 가 금융 숫자를 직접 추론하면 재현성과 신뢰성이 낮다. 따라서 계산은 검증된 엔진이 하고 AI 는 오케스트레이션과 설명에 집중해야 한다.

2.4 프로젝트의 차별화

일반 대출/자산관리 서비스	LoanOps AI
현재 대출한도·금리 조회	결정 전후의 Financial Twin 을 비교
상품 추천 중심	현금흐름·유동성·부채부담까지 통합 분석
한 번의 조회	금리/소득 shock 에 대한 Stress Test
LLM 답변	Deterministic 계산 → Agent Tool Calling → 설명
결과 숫자 중심	가정·공식·데이터 출처·계산 버전을 추적

3. 사용용도와 대상 사용자

주요 금융 의사결정을 “사전에 시뮬레이션”하는 개인 금융 의사결정 도구.

3.1 주요 대상 사용자

Persona	상황	LoanOps AI 가 주는 가치
첫 주택구매 직장인	주택가격·대출규모·자기자본을 함께 판단해야 함	구매 전후 월 현금흐름과 비상자금 변화를 즉시 계산
다중대출 보유자	여러 대출의 금리·상환부담이 분산됨	전체 부채를 합산해 대환/선상환 시나리오 비교
재무여력이 불확실한 사용자	소득은 충분해 보여도 지출과 유동성이 불안정	Stress Test 와 Cashflow 분석으로 취약 구간 확인
금융정보를 어려워하는 사용자	DSR, 금리, 상환방식 등의 의미를 이해하기 어려움	Agent 가 계산 결과와 근거를 자연어로 설명

3.2 핵심 사용용도

- 주택구매 시뮬레이션: 목표 주택가격과 자기자본을 입력하면 예상 대출규모, 월 상환액, 구매 후 현금흐름을 계산한다.
- 신규대출 시뮬레이션: 기존 부채와 소득을 반영해 신규대출이 부채부담과 월 여유자금에 주는 영향을 계산한다.
- Stress Test: 금리 +1/+2/+3%p, 소득 -10/-20/-30% 등의 충격을 적용해 재무상태의 내구성을 본다.
- 대환 검토 신호: 보유 대출금리와 LoanOps Serving 상품 데이터를 비교해 “재검토할 가치가 있는 상황”을 탐지한다.
- 금융상품 매칭: 필요한 대출규모와 월 상환 여력, 기간·상환방식, LoanOps 상품조건을 결합해 검토 후보를 추리고 상품별 월 상환액·총이자·Stress 결과를 동일 기준으로 비교한다.
- 유동성 모니터링: 카드대금 증가, 현금성자산 감소, 고정비 상승 등으로 비상자금 기준이 깨지는 시점을 감지한다.
- 대안 비교: 더 저렴한 주택, 기존대출 선상환, 구매시점 연기 등 복수의 What-if 시나리오를 동일 기준으로 비교한다.

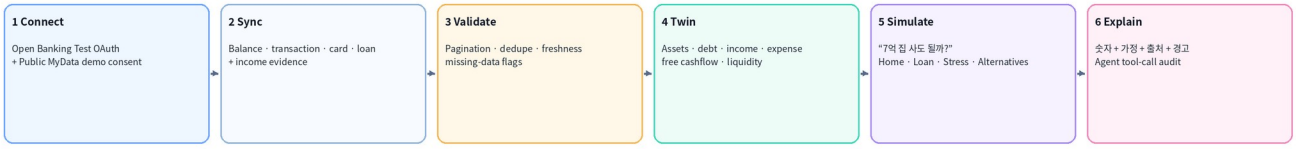
3.3 서비스가 하지 않는 것

중요 경계

은행의 실제 대출승인, 확정 한도, 투자수익을 보장하지 않는다. 화면에는 “예상/시뮬레이션 값”임을 명확히 표시하고, 실제 심사기준·담보평가·개인 신용정책 등 외부 요인이 존재함을 고지한다.

4. 사용방법 - User Journey

사용자는 금융데이터를 연결하고, 자연어로 목표를 말하고, 시나리오 결과와 근거를 확인한다.



4.1 최초 사용

1. 데이터 연결: Open Banking 개발/테스트 환경을 OAuth 2.0 방식으로 연결하고, 공공 MyData Demo Consent 를 통해 소득·재직·납세 근거를 불러온다. 포트폴리오 MVP 의 공공 MyData payload 는 공식 구조에 맞춘 Mock Provider 를 사용한다.
2. 동의 범위 확인: 계좌/거래/카드/대출 등 Open Banking scope 와 소득·재직 등 공공 MyData 항목 중 어떤 정보가 분석에 사용되는지 화면에서 확인한다.
3. Financial Twin 생성: 자산, 부채, 월 소득, 월 지출, 원리금 상환, 유동성 지표를 계산한다.
4. Dashboard 에서 현재 상태를 확인한다: Net Worth, Monthly Free Cashflow, Debt, Emergency Fund 등을 표시한다.

4.2 대표 사용 시나리오



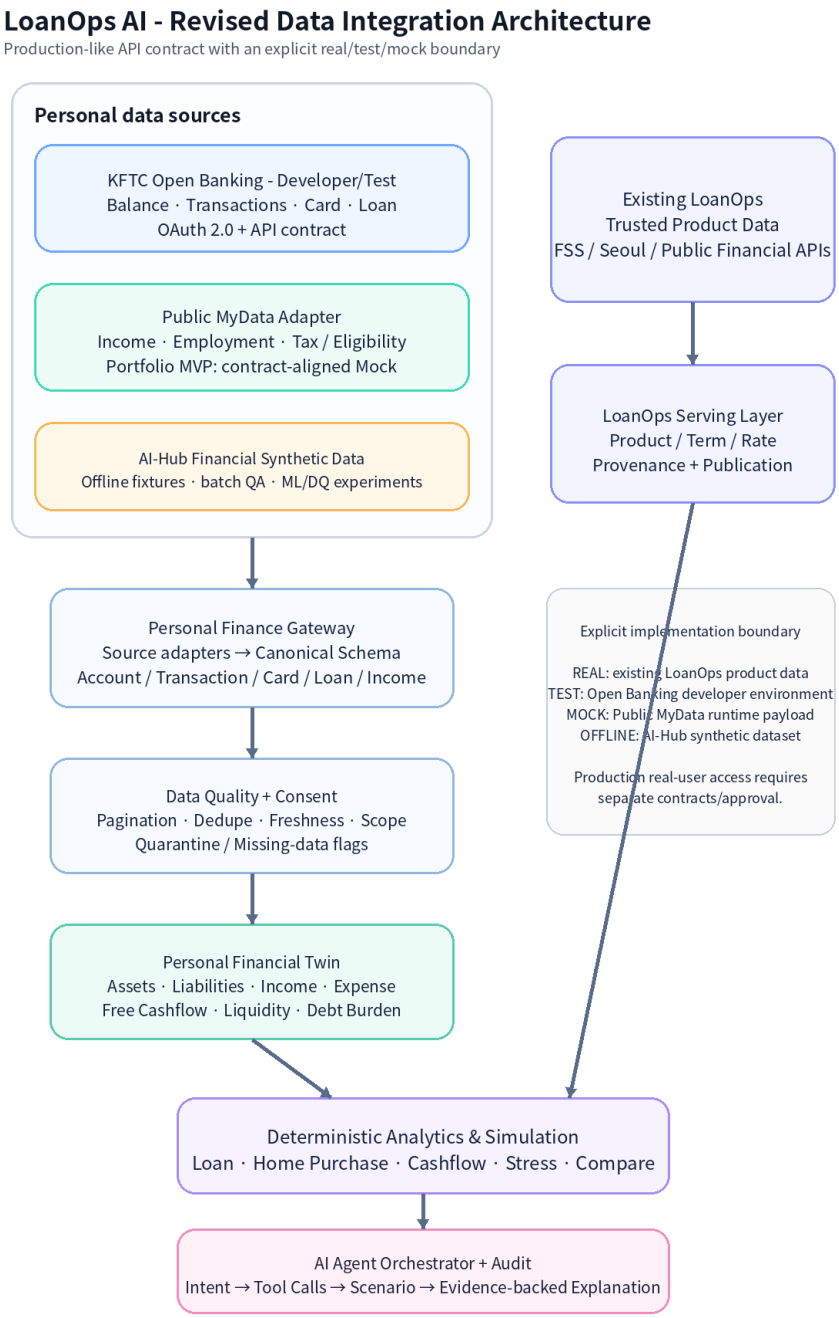
4.3 Proactive 사용

사용자가 매번 질문하지 않아도 데이터가 갱신될 때 Opportunity Engine 이 중요한 변화를 감지한다. 예를 들어 신용도와 소득이 개선되었는데 기존 대출금리가 상대적으로 높거나, 카드대금 급증으로 비상자금 기준이 무너질 것으로 보이면 “검토할 금융 이벤트”로 알린다.

감지 변화	Event	사용자에게 보이는 메시지 예시
소득 증가 + 기존부채 감소	RATE_REVIEW	“현재 조건 변화로 기존 대출조건을 재검토할 가치가 있습니다.”
현재 대출금리 > 비교상품 범위	REFINANCING_REVIEW	“비교대상 금리와 차이가 커졌습니다. 대환 가능 여부를 확인해 보세요.”
Free Cashflow 급감	LIQUIDITY_RISK	“현재 지출패턴이 유지되면 비상자금 기준을 하회할 수 있습니다.”

5. 시스템 아키텍처

Existing LoanOps 를 Trusted Product Data Layer 로 유지하고, 그 위에 Personal Finance 와 Simulation 을 쌓는다.



5.1 설계 원칙

- 기존 LoanOps 수집/정규화/품질/서빙 파이프라인은 수정 최소화. 신규 기능은 별도 도메인 모듈로 추가한다.
- 개인 금융데이터와 상품 데이터를 저장 계층부터 분리한다. Open Banking/Public MyData 계열은 Personal Finance Gateway 로, 기존 LoanOps 는 Product Serving Layer 로 유지하고 Simulation 단계에서만 명시적으로 결합한다.
- LLM 은 금리·상환액·DSR 등 금융 수치를 직접 계산하지 않는다. 계산 Tool 의 결과만 설명한다.
- 모든 시뮬레이션은 재현 가능해야 한다. 입력, 가정, 규칙 버전, 데이터 스냅샷, 결과를 기록한다.

- 규제/계산규칙은 코드에 하드코딩하기보다 Versioned Policy/Rule Layer 로 분리할 수 있게 설계한다.

5.2 추천 백엔드 패키지 구조

```
com.loanops
├── ingestion
│   ├── fss / seoul
│   ├── openbanking      # OAuth / balance / transaction / card / loan
│   ├── publicmydata     # official-contract adapter / mock
│   └── aihub_fixture    # offline dataset loader
├── normalization
├── quality
├── publication
├── product              # existing trusted product domain
├── personal             # account / transaction / loan / income / asset
├── analytics            # cashflow / debt / liquidity / wealth
├── simulation           # loan / home / stress / scenario
├── opportunity
├── agent                # tools / orchestration / audit
└── api
```

5.3 왜 Modular Monolith 인가

포트폴리오 단계에서는 microservice 로 분리하는 것보다 Spring Boot Modular Monolith 가 적합하다. 도메인 경계를 명확히 유지하면서도 트랜잭션, 로컬 개발, 테스트, 배포 복잡도를 낮출 수 있다. 서비스 사용량과 팀 규모가 커질 때 ingestion, simulation, agent 등을 독립 서비스로 분리할 수 있게 인터페이스만 안정적으로 설계한다.

6. Personal Financial Data & Financial Twin

마이데이터의 원천 레코드를 “현재 재무상태”로 변환하는 분석 계층.

6.1 Provider 설계

Personal Finance Provider 는 “실제/테스트/Mock” 경계를 명확히 분리한다. Open Banking 은 금융결제원 개발자 환경의 OAuth/API 계약을 직접 연동하고, Public MyData 는 이용기관 승인·현장실사 등 별도 절차가 필요한 점을 반영해 공식 묶음정보 구조에 맞춘 Mock Adapter 로 구현한다. AI-Hub 금융 합성 데이터는 런타임 사용자 데이터가 아니라 대규모 테스트·품질검증·분석 실험용 offline fixture 로 사용한다.

```
PersonalFinanceProvider
  - capabilities()
  - getAccounts(userId)
  - getBalances(userId)
  - getTransactions(userId, from, to)
  - getLoans(userId)
  - getCards(userId)
  - getIncomeEvidence(userId)

Implementations
  - OpenBankingProvider          # developer/test API contract
  - MockPublicMyDataProvider     # official structure, demo payload
  - AiHubFixtureProvider         # offline test only
  - CsvFallbackProvider          # deterministic local fallback
  - RealPublicMyDataProvider     # future, after approval
```

6.2 주요 데이터 모델

Entity / Table	핵심 내용
financial_user	사용자 식별 및 profile
consent_grant	데이터 이용 동의 범위/기간
financial_account	은행/증권 계좌
transaction	입출금 및 카드 거래
loan_account	실제 보유 대출, 금리, 잔액, 상환방식
loan_payment	원금/이자 납입 이력
investment_position	투자자산 보유현황
balance_snapshot	시점별 계좌/자산 잔액
financial_snapshot	특정 시점의 통합 Financial Twin
financial_metric	순자산, 여유현금흐름, 비상자금 등 파생지표
scenario / result	What-if 입력 가정과 결과
opportunity_event	감지된 금융 이벤트
agent_run / tool_call	Agent 실행 및 Tool 사용 추적

6.3 Financial Twin 핵심 지표

지표	계산 개념	의사결정에서의 의미
Net Worth	총자산 - 총부채	현재 순재산의 기준점
Monthly Free Cashflow	월소득 - 생활비 - 원리금 등 고정지출	새로운 대출을 감당할 여력
Emergency Fund Months	현금성자산 / 월 필수지출	소득충격에 버틸 수 있는 기간
Debt Service Burden	연간 원리금상환 / 연간소득	부채 상환 부담
Debt-to-Asset Ratio	총부채 / 총자산	자산 대비 레버리지
Liquid Asset Ratio	현금성자산 / 총자산	즉시 사용할 수 있는 유동성

6.4 실제 데이터 소스 전략 - 무엇을 어디까지 "진짜"로 구현할 것인가

MVP 의 핵심은 모든 데이터를 억지로 실데이터로 만드는 것이 아니라, 실제 금융 API 계약을 직접 연동하는 영역과 승인 때문에 Mock 이 필요한 영역을 명확히 분리하는 것이다. 이렇게 해야 데모의 신뢰도가 올라가고 향후 production 전환도 Adapter 교체만으로 가능하다.

Source	MVP 에서의 역할	실제성 수준	런타임 사용
KFTC Open Banking	계좌 잔액, 거래내역, 카드/대출 capability 연동. OAuth 2.0, token, pagination, API error handling 을 실제 계약으로 구현	Developer/Test API 계약 직접 연동	Yes - Test
Public MyData	소득·재직·납세/자격 같은 대출 판단 보조 evidence	공식 묶음정보/API 구조에 맞춘 Mock Provider. 실제 이용은 기관 승인 필요	Yes - Mock
Existing LoanOps	비교 가능한 대출상품·금리 및 publication/provenance	현재 구현된 실제 공공 금융 데이터 파이프라인	Yes - Real product data
AI-Hub 금융 합성 데이터	대규모 fixture, data quality stress, feature/ML 실험	실제 금융기관 특성을 학습해 만든 synthetic dataset	No - Offline
User Input	목표 주택가격, 계약 예정금, 기간, 계획금리 등 scenario assumption	사용자 명시 입력	Yes

핵심 원칙

Open Banking 의 테스트 데이터를 "실제 고객 마이데이터"라고 표현하지 않는다. 공공 MyData Mock 도 "실연동"이라고 표현하지 않는다. 대신 실제 API 계약/OAuth/데이터 모델/검증 흐름을 production-equivalent 하게 구현하고, 각 결과 화면에 source type 을 노출한다.

6.5 Open Banking 개발/테스트 환경을 런타임 Personal Data Backbone 으로 사용

금융결제원 개발자 사이트는 금융 Open API 에 관심 있는 일반 개발자도 가입할 수 있고, 서비스 신청·API Key·테스트 정보 등록 후 REST API 테스트를 수행할 수 있다. LoanOps AI 에서는 이를 "금융기관 실데이터 대체용 dataset"이 아니라 "실제 Open Banking API 계약을 검증하는 개발 환경"으로 사용한다.

구현 순서는 아래와 같이 고정한다.

- 개발자 계정/애플리케이션 등록 → client 정보와 redirect URI 를 환경변수/Secret 로 관리.
- OAuth 2.0 인증 흐름을 구현하고 Access Token/Refresh Token 의 발급·갱신·만료를 TokenService 로 캡슐화.
- 사용자별 fintech_use_num 또는 테스트 식별자를 Provider 내부에서 관리하고 도메인 계층에는 노출하지 않음.
- 잔액조회 API 를 첫 PoC 로 연결한 뒤 거래내역조회 API 를 추가. 거래내역은 페이지당 최대 25 건이므로 pagination cursor/next-page 처리를 ingestion 책임으로 둠.
- 카드/대출·리스 API 는 Open Banking 서비스 범위에는 존재하지만, 개발자 환경에서 실제 사용 가능한 capability 를 확인한 뒤 optional adapter 로 활성화. 지원되지 않으면 동일 canonical contract 의 deterministic fixture 로 fallback.
- 모든 응답은 먼저 raw_openbanking_snapshot 에 원문/수집시각/hash 를 남긴 뒤 canonical model 로 정규화.

기능	공식 API/계약 예시	LoanOps AI 구현 포인트
OAuth	OAuth 2.0 기반 사용자 인증	token lifecycle, scope, redirect, refresh 실패 처리
잔액	GET /v2.0/account/balance/fin_num	현재 liquid balance snapshot 생성
거래내역	GET /v2.0/account/transaction_list/fin_num	25 건/page pagination, 기간 sync, dedupe
카드	카드목록/기본/청구 API 가 서비스 범위에 포함	capability 가 열릴 때 card_account/card_bill 로 매핑
대출·리스	목록/기본정보 API 가 서비스 범위에 포함	loan_account, repayment metadata 로 매핑

6.6 Public MyData 는 "공식 계약을 따르는 Mock Adapter"로 구현

공공 MyData 는 일반 data.go.kr Open API 처럼 개인이 API Key 만 받아 자신의 소득·재직정보를 호출하는 구조가 아니다. 이용기관 신청서, 이용환경 조사표, 묶음정보 신청, 업무구조도 제출, 현장실사와 심사 등 별도 절차가 있다. 따라서 포트폴리오 단계에서는 실제 이용기관 행세를 하지 않고, 공식 가이드의 묶음정보/API 흐름을 반영한 MockPublicMyDataProvider 를 만든다.

- 런타임에는 연소득, 소득 기준일, 재직/건강보험 자격 상태, evidence type, source/as-of 를 반환.
- 원본 주민등록번호, 상세 행정문서 이미지 등 불필요한 민감정보는 Demo DB 에 저장하지 않음.
- UI 에는 "Public MyData Demo Evidence"라고 표시하고 실제 기관 조회가 아님을 명확히 표시.
- 향후 승인된 기관 환경에서는 RealPublicMyDataProvider 가 동일 interface 를 구현하도록 하여 분석/시뮬레이션 코드는 변경하지 않음.

```
IncomeEvidence {
  verifiedAnnualIncome
  incomeYear
  employmentStatus
  evidenceType      // INCOME_CERTIFICATE / ELIGIBILITY
  asOf
  sourceType        // PUBLIC_MYDATA_MOCK | PUBLIC_MYDATA_REAL
}
```

6.7 AI-Hub 금융 합성 데이터의 정확한 활용 위치

AI-Hub 금융 합성 데이터는 카드·CB·금융상품 등 대규모 panel 을 제공하므로 "Financial Twin 런타임 사용자"로 직접 섞기보다 offline test/analysis layer 에 두는 것이 안전하다. 서로 다른 출처의 한 레코드를 동일 개인으로 임의 join 하면 데이터 의미가 깨지므로 금지한다.

- Data Quality rule stress: 누락/극단값/고부채/카드사용 급증 분포를 참고해 synthetic fixture 생성.
- Cashflow/위험 feature 실험: ML 을 추가할 경우 train/evaluation 데이터로만 사용.
- Persona generator calibration: Demo persona 가 지나치게 비현실적인 값이 되지 않도록 분포 참고.
- API E2E 테스트는 Open Banking test fixture 를 사용하고, AI-Hub batch 분석은 별도 job 으로 분리.

6.8 Canonical Personal Finance Schema - 소스가 달라도 같은 모델로 정규화

Canonical Entity	주요 필드	Primary Source	Fallback / Derived
financial_account	institution, account_type, currency, status	Open Banking	CSV fixture
balance_snapshot	account_id, balance, available_balance, as_of	Open Banking balance	Fixture
transaction	txn_id, date, amount, direction, merchant/category, balance_after	Open Banking transaction	AI-Hub-derived fixture
card_account / card_bill	issuer, billing_date, billed_amount, payment_account	Open Banking capability	Fixture
loan_account	institution, principal_balance, repayment_type, payment_day, next_interest_date	Open Banking loan/lease capability	Fixture

Canonical Entity	주요 필드	Primary Source	Fallback / Derived
income_evidence	verified_income, income_year, employment_status, evidence_type	Public MyData Mock/Real	User input + transaction inference
financial_snapshot	assets, liabilities, income, expense, free_cashflow, emergency_months	Derived	-

중요한 점은 "연소득"과 "월평균 입금"을 같은 값으로 취급하지 않는 것이다. Public MyData evidence 는 verified income, Open Banking 거래내역에서 추론한 급여성 입금은 observed cashflow 로 별도 저장하고 Simulation 이 어떤 값을 사용했는지 rule_version 과 함께 기록한다.

6.9 동기화, 데이터 품질, 실패처리

- Incremental sync: last_successful_sync_at 이후 거래만 요청하되, 중복/지연 반영을 고려해 일정 overlap window 를 재조회하고 txn key/hash 로 dedupe.
- Pagination completeness: 거래내역의 페이지당 제한을 고려해 "마지막 페이지까지 수집 완료" 플래그가 없으면 Financial Twin 을 완전한 상태로 승격하지 않음.
- Freshness: balance/loan/card/income evidence 마다 freshness SLA 를 별도로 두고 stale 데이터는 UI warning 으로 노출.
- Consistency: 대출잔액과 최근 상환 거래가 비정상적으로 불일치하면 warning 또는 quarantine. 계산에는 마지막 정상 snapshot 을 사용.
- Failure isolation: 한 기관/endpoint 실패가 전체 sync 를 막지 않게 source_sync_run 단위로 status 를 저장.
- Provenance: 모든 Financial Snapshot metric 은 source_record_id 또는 evidence_id 까지 역추적 가능하게 구성.

검증 Rule	예시	처리
DUPLICATE_TRANSACTION	같은 거래 key/hash 반복 수신	중복 제거 + count 기록
PAGINATION_INCOMPLETE	next page 존재하나 수집 중 실패	snapshot 승격 보류
STALE_BALANCE	마지막 잔액 시각이 기준 초과	warning + stress 계산 제한
MISSING_INCOME_EVIDENCE	verified income 없음	사용자 입력/관측소득 fallback, 결과에 경고
LOAN_PAYMENT_MISMATCH	대출 metadata 와 관측 상환패턴 불일치	manual review flag
SOURCE_CAPABILITY_MISSING	card/loan endpoint 미지원	deterministic fixture 또는 기능 비활성화

6.10 "실제 구현"과 "데모용 Mock"의 경계를 제품 기능으로 노출

UI Badge	의미	예시
REAL PRODUCT DATA	기존 LoanOps 가 실제 외부 공공 금융 소스에서 수집·검증한 상품정보	상품 금리/기관/기준일
OPEN BANKING TEST	금융결제원 개발/테스트 API 계약을 통해 들어온 테스트 데이터	계좌잔액/거래내역
PUBLIC MYDATA MOCK	공식 구조를 따르지만 포트폴리오에서 직접 행정기관 조회한 것은 아닌 데이터	연소득/재직 evidence
USER INPUT	사용자가 scenario 를 위해 직접 입력한 가정	주택가격/계약금
DERIVED	위 데이터에서 deterministic rule 로 계산한 지표	Free Cashflow/Stress result

이 경계가 포트폴리오의 강점

"전부 실데이터"라고 과장하는 대신, 실제 API 계약을 연동한 부분과 승인 때문에 Mock 한 부분을 기술적으로 분리하고 source badge·audit trace 로 노출한다. 금융 AI 에서 중요한 신뢰성/컴플라이언스 감각을 보여줄 수 있다.

7. Core Simulation Engines

프로젝트의 “금융 AI”는 LLM 보다 이 계산 엔진이 중심이다.

7.1 Loan Simulator

신규대출 금액, 예상금리, 기간, 상환방식을 입력받아 월 상환액과 기존 대출을 포함한 총 부채부담을 계산한다. 실제 승인한도가 아니라 “입력 가정 기준 예상 결과”로 제공한다.

Input	Calculation	Output
대출금액, 금리, 기간, 상환방식	원리금 상환스케줄	월 상환액, 총이자, 잔액추이
현재 소득, 기존 대출	기존+신규 부채부담	Estimated debt-service ratio
LoanOps Serving data	유사 상품 조건 비교	현재 가정이 시장 데이터에서 어느 위치인지

7.2 Home Purchase Simulator

주택가격, 자기자본, 신규 주담대, 부대비용 등을 적용해 “구매 전 Financial Twin”과 “구매 후 Financial Twin”을 비교한다. 프로젝트의 대표 사용자 가치가 가장 선명하게 보이는 기능이다.

```
Before
  Cash / Investments / Existing Debt / Free Cashflow
                ↓   home purchase scenario
After
  House Asset / Reduced Cash / New Mortgage / New Monthly Payment
                ↓
Change in Liquidity, Debt Burden, Emergency Fund, Monthly Free Cashflow
```

7.3 Stress Test Engine

Base Scenario 에 여러 shock 을 적용한다. 초기에는 deterministic grid 방식이 가장 명확하다. Monte Carlo 는 MVP 이후 확장 기능으로 둔다.

Shock	예시	관찰 결과
Interest Rate	+1 / +2 / +3%p	월 상환액, Free Cashflow 감소
Income	-10 / -20 / -30%	현금흐름 적자 전환 여부
Expense Inflation	필수지출 +10 / +20%	비상자금 소진속도
Asset Drawdown	투자자산 -20 / -30%	유동성/순자산 감소
Combined	금리 +2%p + 소득 -20%	복합위험에서 버틸 수 있는 기간

7.4 Alternative Scenario Comparison

사용자가 “그럼 어떻게 하면 돼?”라고 물으면 Agent 가 조언을 즉흥 생성하는 대신, 미리 정의된 시나리오 템플릿을 사용해 여러 대안을 실제 엔진으로 계산한 뒤 비교한다.

Scenario	변경 가정	비교 목적
A. 현재 구매	기준안	Base
B. 가격 하향	주택가격 -5 천만원	대출규모 감소 효과

Scenario	변경 가정	비교 목적
C. 기존대출 선상환	신용대출 일부/전액 상환	부채부담 감소 효과
D. 구매 연기	12 개월 저축 반영	자기자본 확대 효과

8. Existing LoanOps 와의 구체적 결합

기존 프로젝트의 데이터 인프라가 AI 결과의 신뢰 근거가 된다.

8.1 역할 변화

LoanOps v1 구성	LoanOps AI 에서의 역할
RawSnapshot	금융상품 원본과 수집시점의 근거
SourceNormalizationService	여러 소스를 동일한 상품/금리 모델로 변환
InstitutionResolver	기관명·코드 정합성 유지
LoanProduct / ProductTerm	Simulation 에 사용할 상품 조건
LoanRateObservation	금리 변화/비교 기준
QualityValidationService	Agent 가 참조해도 되는 데이터인지 판정
QuarantineRecord	오류/의심 데이터가 의사결정 계층으로 전달되지 않게 차단
Publication / Serving	검증 완료 데이터만 Analytics/Agent 에 노출
DataChange	상품 조건 변화 → Opportunity/Event 재평가 트리거

8.2 대환/금리 재검토 Event 예시

```
Personal Twin
  Current loan rate = 6.1%
  Remaining balance = 30M KRW
          +
LoanOps Serving Products
  comparable observed rates around 5.x%
          ↓
Opportunity Engine
  REFINANCING_REVIEW
          ↓
User Message
  '비교대상 금리와 차이가 커졌습니다. 실제 대환 가능 여부를 확인할 가치가 있습니다.'
```

이때 시스템은 특정 금융상품 가입을 단정적으로 권하지 않는다. LoanOps 의 데이터는 “재검토할 필요가 있는지”를 판단하는 근거로 사용하고, 실제 가입/승인 가능성은 별도 금융기관 심사가 필요하다고 표시한다.

8.3 Traceability 를 제품 기능으로 만들기

기존 LoanOps 의 provenance 철학을 AI 화면까지 이어간다. 사용자는 “예상 월 상환액”이나 “비교 금리” 옆의 계산근거를 눌러 어떤 데이터 스냅샷, 어떤 공식, 어떤 가정, 어떤 상품 데이터 버전이 사용되었는지 확인할 수 있다.

8.4 Product Matching Engine - 상품 “추천”보다 근거 있는 후보 매칭

MVP 에서는 대출상품만 다룬다. 시스템은 특정 상품을 무조건 가입하라고 권하는 대신, 현재 Financial Twin 과 사용자의 금융 목표를 기준으로 “검토할 가치가 있는 후보”를 좁힌다. 핵심은 금리순 정렬이 아니라 후보마다 실제 월 상환액, 총 이자비용, 구매 후 잉여현금, 금리상승 스트레스 결과까지 다시 계산하는 것이다.

단계	입력/판단	출력
1. Goal Parsing	주택구매/대환/신규대출 목적, 필요금액, 목표시점, 최대 월 상환액, 선호기간·상환방식	Product Match Request
2. Capacity Envelope	Financial Twin 의 소득·기존부채·여유현금·비상자금과 시나리오 규칙	예상 대출규모 범위와 월 상환 예산
3. Candidate Retrieval	ServingLoanProduct/Term/RateObservation 에서 목적·금액·기간이 관련된 상품 조회	Raw Candidate Pool
4. Hard Filter	명시된 한도, 기간, 상환방식, 목적, 확인 가능한 자격조건만 적용	검토 가능 후보 / 제외 / 정보부족
5. Candidate Simulation	후보별 금리·기간·상환방식을 Loan Simulator 와 Home Simulator 에 투입	월 상환액, 총비용, 구매 후 Financial Twin
6. Stress Evaluation	금리 +1/+2%p, 소득감소 등 동일 shock 적용	후보별 resilience metrics
7. Deterministic Ranking	Affordability, 비용, stress, 선호적합도, 데이터 신선도 점수화	Top-K 검토 후보
8. Agent Explanation	점수·시뮬레이션·상품출처만 사용해 설명	왜 후보인지 / 무엇을 확인해야 하는지

중요: Hard Filter 에서 확인되지 않은 자격조건을 LLM 이 추정하지 않는다. LoanOps 데이터에 가입조건이나 한도 필드가 없으면 후보 상태를 MATCHED_FOR_REVIEW 가 아니라 INSUFFICIENT_DATA 로 표시하거나 “추가 확인 필요” 경고를 붙인다.

8.5 LoanOps 상품 데이터에 추가/강화할 필드

현재 LoanOps 의 상품·조건·금리 관측 구조를 그대로 활용하되, 사용자 매칭을 위해 다음 필드를 canonical product view 에 명시한다. 값이 원천에서 제공되지 않으면 NULL 을 보존하고 임의 추정하지 않는다.

필드	용도	결측 시 처리
productType / purpose	주담대·신용대출 등 목적별 후보 retrieval	목적 기반 자동매칭 제외
institution / productName	사용자에게 후보 식별	필수
minRate / maxRate / rateType	월 상환·총이자·Stress 계산	관측금리 범위로만 표시, 확정금리 아님
termMin / termMax	사용자 목표기간 적합성	기간 필터 대신 확인 필요
repaymentMethods	원리금균등/원금균등/만기일시 등 계산방식 결정	지원 방식 불명확 시 시뮬레이션 제한
amountMin / amountMax	필요 대출액이 상품 범위에 드는지 확인	한도 적합성 점수 제외
eligibilityText / constraints	가입대상·우대·주택조건 등 설명 가능한 제한	LLM 추정 금지, 사용자 확인 항목으로 노출
source / observedAt / publicationId	데이터 provenance 와 freshness	stale 이면 ranking penalty 또는 후보 제외
qualityStatus	품질검증 통과 여부	Serving 승격 실패 상품은 매칭 대상 제외
match_loan_products()	Financial Twin + 목표 시나리오로 LoanOps 상품의 hard filter 와 Top-K 매칭 실행	
simulate_product_candidate()	특정 후보 상품을 동일 Loan/Home Scenario 에 적용해 월 상환·현금흐름 계산	
compare_product_candidates()	후보별 affordability/cost/stress 지표를 동일 정의로 비교	
get_product_match_trace()	후보 상태, score breakdown, product publication, rule version 추적	

후보 상태는 MATCHED_FOR_REVIEW / FILTERED_OUT / INSUFFICIENT_DATA 세 단계로 단순화한다. “승인 가능”, “가입 가능” 같은 표현은 금융기관 심사를 대신하는 오해를 줄 수 있으므로 사용하지 않는다.

8.6 MVP Matching Score - 금리순이 아니라 “감당 가능성” 중심

Hard Filter 를 통과한 후보에만 0-100 deterministic score 를 계산한다. 아래 가중치는 초기 MVP 예시이며 설정파일과 scoreRuleVersion 으로 버전 관리한다. 사용자는 후보별 세부 점수를 볼 수 있어야 한다.

항목	가중치	계산 개념
Affordability Fit	35%	후보 적용 후 월 상환액과 사용자 최대 월 상환 예산, Free Cashflow 여유를 비교
Interest Cost	25%	동일 금액·기간 기준 예상 월 상환액과 총 이자비용의 상대적 수준
Stress Resilience	20%	금리 +2%p 또는 소득감소 시 Free Cashflow/비상자금이 얼마나 유지되는지
Preference Fit	10%	선호기간, 상환방식, 변동/고정금리 선호와의 일치도
Data Quality & Freshness	10%	Serving quality status, observedAt/publication age, 필수필드 완전성

Score 는 “금융적으로 가장 좋은 상품”의 객관적 순위가 아니라, 사용자가 입력한 목표·제약과 시스템이 계산한 재무여력에 얼마나 잘 맞는지를 나타내는 매칭 점수다. 원점수, 가중치, 규칙 버전을 모두 저장한다.

8.7 상품별 시뮬레이션 예시 - 같은 3.6 억원이어도 결과가 다르다

아래 수치는 기능 설명을 위한 synthetic example 이다. 사용자가 7 억원 주택 구매를 위해 3.6 억원 주담대가 필요하고, 월 상환 예산을 약 180 만원으로 설정했다고 가정한다.

후보	가정 금리/기간	예상 월 상환액	금리 +2%p 시	매칭 해석
Candidate A	3.8% / 30 년	약 168 만원	약 211 만원	Base affordability 우수, stress 시 월부담 증가 확인 필요
Candidate B	4.0% / 30 년	약 172 만원	약 216 만원	월 상환 예산 내, A 보다 비용은 높지만 다른 조건 비교 필요
Candidate C	4.2% / 30 년	약 176 만원	약 220 만원	Base 는 예산 내이나 stress 여유가 가장 작음

Agent 는 “A 가 최고”라고 결론내리는 대신, 각 후보가 왜 남았는지와 무엇이 trade-off 인지 설명한다. 실제 적용금리는 개인 심사·우대조건·시점에 따라 달라질 수 있으므로 LoanOps 관측금리는 비교/시뮬레이션 입력으로만 사용한다.

8.8 Product Matching 데이터 모델 및 API

매칭 자체도 Scenario 처럼 재현 가능해야 한다. 한 번의 매칭 요청에 어떤 Financial Snapshot, 상품 publication, rule version 이 사용됐는지 저장한다.

엔티티/API	핵심 내용
product_match_run	userId, scenarioId, financialSnapshotId, productPublicationId, scoreRuleVersion, requestedAt
product_match_candidate	productId, candidateStatus, hardFilterReasons, totalScore, rankForReview
candidate_score_component	AFFORDABILITY/COST/STRESS/PREFERENCE/DATA_QUALITY 별 rawScore, weight, weightedScore
candidate_scenario_result	월 상환액, 총이자, 구매 후 Free Cashflow, Emergency Fund, stress results
POST /api/v1/loan-product-matches	scenarioId 또는 goal + amount + monthlyBudget + preferences 로 후보 생성
GET /api/v1/loan-product-matches/{id}	Top-K 후보, 점수 구성, 근거, 데이터출처, warning 반환

응답에는 productId 만 반환하지 않고 productSourceAsOf, rateObservedAt, financialSnapshotId, calculationRuleVersion, scoreRuleVersion, limitations 를 반드시 포함한다. 이 메타데이터가 상품 매칭의 신뢰성과 재현성을 만든다.

8.9 Product Matching Guardrails

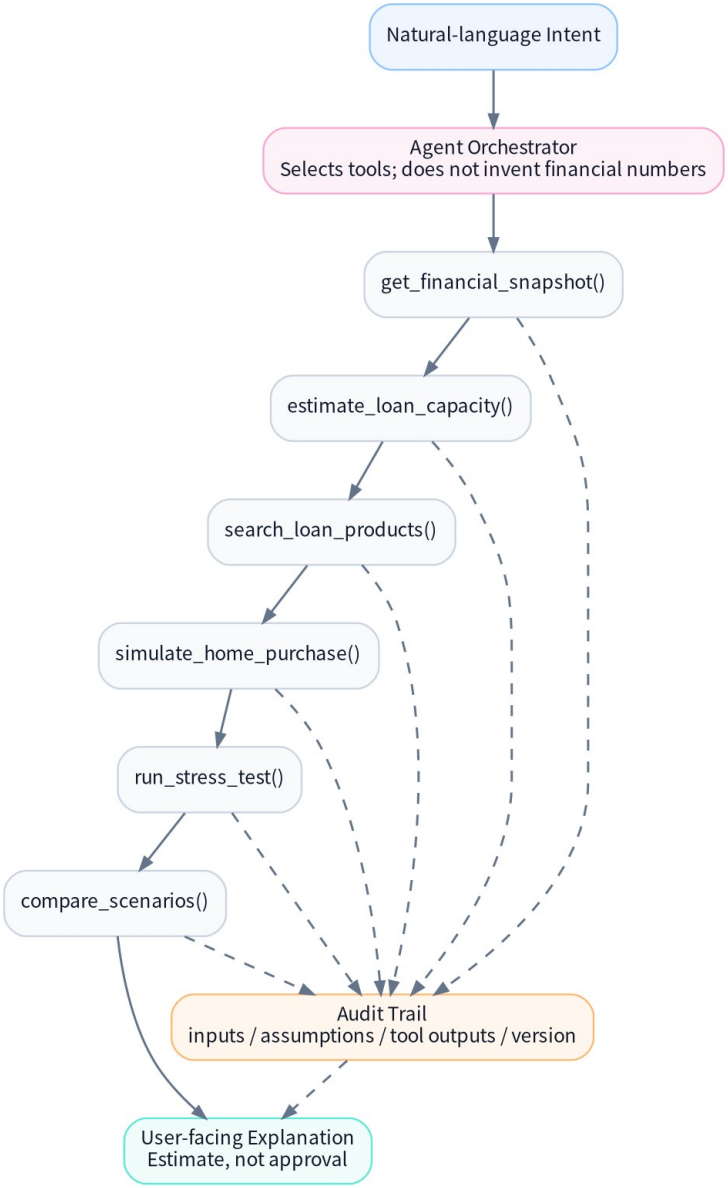
- Serving/Quality Validation 을 통과하지 않은 상품은 후보 풀에 들어오지 않는다.
- 자격조건이 데이터에 없으면 LLM 이 사용자의 가입 가능성을 추정하지 않는다.
- 실제 적용금리·한도·승인 여부는 금융기관 심사 결과와 다를 수 있음을 후보 카드에 명시한다.
- Top-K 는 검토 편의를 위한 매칭 순서이며 “최고/최적 금융상품”이라는 표현은 사용하지 않는다.
- 추천 근거에는 반드시 Financial Twin 지표, 적용한 가정, 상품 데이터 기준일, score breakdown 을 연결한다.

제품 메시지

Trusted Data → Deterministic Analytics → Reproducible Simulation → Traceable AI Explanation

9. AI Agent 설계

AI 는 금융계산기가 아니라 “도구를 선택하고 결과를 설명하는 오케스트레이터”다.


Natural-language Intent
Agent Orchestrator
Selects tools; does not invent financial numbers
get_financial_snapshot()
estimate_loan_capacity()
search_loan_products()
simulate_home_purchase()
run_stress_test()
compare_scenarios()
Audit Trail
inputs / assumptions / tool outputs / version
User-facing Explanation
Estimate, not approval

9.1 제공 Tool

Tool	역할
get_financial_snapshot()	현재 Financial Twin 조회
get_cashflow_analysis()	소득/지출/고정비/여유현금 분석
get_debt_analysis()	보유 대출·상환부담 집계
search_loan_products()	LoanOps Serving 데이터 검색
estimate_loan_capacity()	입력 가정에 따른 예상 부채여력 계산
simulate_home_purchase()	주택구매 전후 재무상태 계산
run_stress_test()	금리/소득/지출 shock 적용
compare_scenarios()	여러 대안의 동일 지표 비교
get_opportunity_events()	현재 감지된 금융 이벤트 조회
get_calculation_trace()	입력/공식/데이터버전/결과 추적
match_loan_products()	Financial Twin + 목표 시나리오로 LoanOps 상품의 hard filter 와 Top-K 매칭 실행
simulate_product_candidate()	특정 후보 상품을 동일 Loan/Home Scenario 에 적용해 월 상환·현금흐름 계산
compare_product_candidates()	후보별 affordability/cost/stress 지표를 동일 정의로 비교
get_product_match_trace()	후보 상태, score breakdown, product publication, rule version 추적

9.2 Agent Guardrails

- 금리·원리금·재무지표를 LLM 이 자유롭게 계산하지 않게 한다. 반드시 Tool Output 을 사용한다.
- 상품 매칭도 LLM 이 자유롭게 순위를 만들지 않는다. Hard Filter 와 deterministic score 가 먼저 실행되고, Agent 는 결과를 설명하거나 추가 시나리오를 호출한다.
- 데이터가 없으면 숫자를 추정하지 않고 “입력 필요” 또는 “분석 불가”를 반환한다.
- 실제 대출승인이나 확정한도처럼 외부 심사에 의존하는 값은 estimated/simulated 로 표시한다.
- 시나리오 비교에서는 각 결과의 가정과 입력을 함께 노출한다.
- AgentRun 과 ToolCall 을 저장해 동일 입력으로 결과를 재현하고 디버깅할 수 있게 한다.

9.3 Audit 데이터

기록 항목	예시
agent_run	사용자 질문, model/version, 시작/종료시간
tool_call	Tool 명, 파라미터, 결과 hash/status
data_snapshot	Financial Snapshot ID, Product Publication ID
assumption	금리 4.0%, 기간 30 년, 소득 shock -20%
rule_version	loan-calculation-v1, stress-policy-v1
final_explanation	사용자에게 실제 노출된 설명

10. Consumer UI / UX

기존 DataOps 화면은 Admin 으로 남기고, 사용자용 Data Connections + Personal Finance 화면을 새로 만든다. 사용자는 Open Banking Test 연결 상태, 마지막 동기화 시각, Public MyData Demo 근거 사용 여부를 먼저 확인한 뒤 Financial Twin 과 Scenario 화면으로 이동한다.

10.0 Data Connections - 사용자가 데이터 출처와 신뢰도를 먼저 본다

Connection	상태 표시	사용자에게 보여줄 정보
Open Banking Test	Connected / Token Expiring / Sync Failed	scope, last sync, accounts synced, source badge
Public MyData Demo	Demo Evidence Loaded	income year, employment evidence, “Mock” 표시
LoanOps Product Data	Published	publication id, last updated, validation status
AI-Hub Offline	Not a user connection	README/Admin 에서만 dataset/version 표시

10.1 Dashboard

Card	표시값	행동
Net Worth	자산 - 부채	상세 자산구성 보기
Monthly Free Cashflow	최근 월평균 및 추세	지출분석 보기
Debt	총잔액·월 상환액	대출 상세/대환 검토
Emergency Fund	몇 개월치 필수지출인지	Stress Test 실행
Opportunities	감지 Event 개수	이벤트별 분석 실행
6. Product Matching	candidate retrieval, hard filter, deterministic score, 후보별 simulation, provenance	7 억원 Demo 에서 Top-K 후보와 score breakdown 을 동일 입력으로 재현

10.2 Ask LoanOps AI



10.3 Scenario Result 화면

결과 화면은 “현재 vs 의사결정 후” 비교가 첫 화면에 보여야 한다. 숫자를 길게 나열하기보다 Free Cashflow, Debt, Liquid Assets, Emergency Fund 를 핵심 KPI 로 잡고, Stress Test 와 Alternative Scenario 는 탭으로 분리한다. 각 주요 수치에는 “근거 보기”를 제공해 입력값, 가정, 공식, LoanOps 데이터 출처, publication time, scenario ID 를 추적할 수 있게 한다.

10.4 Matched Loan Candidates

Home/Loan Scenario 가 필요한 대출규모를 계산하면 별도 후보 비교 화면을 보여준다. 기본 정렬은 Product Match Score 지만 사용자는 예상 월 상환액, 금리 관측값, Stress 결과로 다시 정렬할 수 있다.

후보 카드에 표시	설명
Match Score + 상태	MATCHED_FOR_REVIEW 또는 INSUFFICIENT_DATA 를 명확히 표시
예상 월 상환액 / 총 이자	동일 금액·기간·상환방식 기준으로 계산한 deterministic 결과
구매 후 Free Cashflow	상품을 적용했을 때 Personal Financial Twin 의 변화
Stress +2%p	금리충격 시 월 상환액과 Free Cashflow 변화
Why matched?	금액·월 예산·기간·선호·데이터 품질 중 어떤 조건이 맞았는지
Evidence	상품 source, observedAt, publicationId, Financial Snapshot, rule version
확인 필요	실제 승인/우대금리/세부 가입조건 등 시스템이 확인하지 못한 항목

Metric	Before	After	Change
Liquid Assets	200M	30M	-170M
Total Debt	30M	420M	+390M
Monthly Free CF	2.35M	0.31M	-2.04M
Emergency Fund	8.4 months	2.7 months	-5.7 months

11. API & Service Contract

Agent 와 프론트가 안정적으로 호출할 deterministic API 를 먼저 완성한다.

Endpoint	Method	목적
/api/v1/financial-snapshots/latest	GET	현재 Financial Twin
/api/v1/analytics/cashflow	GET	현금흐름 분석
/api/v1/analytics/debt	GET	부채 분석
/api/v1/scenarios/loan	POST	신규대출 시뮬레이션
/api/v1/scenarios/home-purchase	POST	주택구매 시뮬레이션
/api/v1/scenarios/{id}/stress-test	POST	Stress Test
/api/v1/scenario-comparisons	POST	여러 시나리오 비교
/api/v1/opportunities	GET	금융 Event
/api/v1/agent/chat	POST	Agent 질문/응답
/api/v1/agent/runs/{id}/trace	GET	Agent 계산근거

11.1 Home Purchase 요청 예시

```
POST /api/v1/scenarios/home-purchase
{
  "housePrice": 700000000,
  "cashContribution": 300000000,
  "mortgageAmount": 400000000,
  "interestRate": 0.04,
  "termYears": 30,
  "repaymentType": "AMORTIZING"
}
```

11.2 응답에 반드시 포함할 metadata

- scenarioId / calculatedAt
- financialSnapshotId
- productPublicationId 또는 사용한 금리 데이터 출처
- assumptions
- calculationRuleVersion
- warnings / limitations

11.3 Loan Product Matching 요청/응답

상품 매칭 API 는 기존 Scenario 결과를 입력으로 받을 수 있게 해 “얼마를 빌려야 하는지”와 “어떤 상품이 맞는지”를 분리한다. 자연어 Agent 요청도 내부적으로 동일 API 를 호출한다.

Request 핵심 필드	예시
scenarioId	SCN-HOME-20260921-001
requiredLoanAmount	360000000
maxMonthlyPayment	1800000
preferredTermYears	30
repaymentPreferences	AMORTIZING
topK	3
Response	matchRunId, candidates[], scoreBreakdown, candidateScenario, warnings, provenance

12. 구현 로드맵

Agent 를 먼저 만들지 않는다. 데이터 → 분석 → 시뮬레이션 → Agent 순서로 간다.

Phase	주요 구현	완료 기준
0. Regression	기존 LoanOps ingestion/quality/serving 테스트 고정	기존 기능 100% 통과
1. Open Banking Integration	개발자 등록, OAuth 2.0, 잔액·거래내역 PoC, capability-based adapter	실제 developer endpoint 계약으로 데이터 수집 성공
2. Personal Canonical Model	Account/Balance/Transaction/Card/Loan/Income 모델 + pagination/dedupe	Open Banking 응답을 canonical schema 로 재현 가능하게 적재
3. Public MyData Adapter	소득·재직·납세/자격 evidence 계약 + Mock Provider	공식 구조와 Mock 의 필드 대응표 및 provenance 저장
4. Financial Twin	Snapshot + 현금흐름/유동성/부채부담 계산	동일 input 에서 동일 Snapshot 생성
5. Simulation	Loan + Home Purchase + Stress + Compare	7 억원 주택 시나리오 golden test 통과
6. Product Matching	candidate retrieval, hard filter, deterministic score, 후보별 simulation, provenance	동일 snapshot/publication 에서 Top-K 와 score breakdown 재현
7. Opportunity	대환검토·유동성·부채 Event rules	데이터 갱신 후 event 생성
8. Agent + Audit	Tool Calling, trace, evidence-backed explanation	숫자 hallucination 없는 E2E
9. Consumer UI	Data Connections/Dashboard/Chat/Scenario/Evidence	데모 영상 가능한 UX

12.1 권장 8 주 스프린트

주차	Backend / Data	Frontend / Demo	검증
1 주	KFTC 개발자 등록/서비스 설정, OAuth/token, 잔액·거래내역 PoC	Data Connections shell	토큰/오류/재시도 smoke test
2 주	pagination, raw snapshot, canonical mapping, DQ/중복제거	연결 상태 + sync history	25 건 페이지 경계, duplicate/freshness test
3 주	Public MyData Mock Adapter + AI-Hub fixture loader	Consent/evidence 화면	source/provenance 및 mock-boundary test
4 주	Financial Twin snapshot + metrics	Financial Twin cards	metric golden test
5 주	Loan/Home Simulator + 기존 LoanOps 상품 연결	Before/After result	상환·시나리오 재현성
6 주	Stress Test + Compare + Opportunity rules	Stress/Alternatives/Opportunity	shock/event boundary test
7 주	Agent tools + orchestration + audit	AI chat + evidence drawer	unsupported-number / tool-selection evaluation
8 주	E2E hardening, security/redaction, README/video	Demo persona 3 종 + main demo	failure cases + demo rehearsal

12.2 Definition of Done - 프로젝트가 "완성"되었다고 볼 기준

Product Matching DoD: 동일 Financial Snapshot 과 상품 publication 에서 동일 Top-K 와 동일 score breakdown 이 재현되고, 각 후보의 월 상환액·Stress 결과가 golden calculation 과 일치하며, 결측 자격조건을 Agent 가 임의 보완하지 않아야 한다.

영역	완료 조건
Open Banking	OAuth/token 흐름과 잔액·거래내역 endpoint E2E 성공. pagination/재시도/만료 token failure case 포함
Public MyData	소득·재직 evidence Mock 이 공식-contract 기반 schema 로 저장되고 sourceType 이 명시됨
Canonical/DQ	raw → canonical → validation → snapshot 전체 trace 가 연결되며 incomplete pagination 은 승격 차단
Financial Twin	3 개 persona 에 대해 asset/debt/income/expense/free cashflow/liquidity 가 재현 가능
Simulation	Loan/Home/Stress/Alternative 결과가 golden case 와 일치하고 rule_version/as-of 기록
Agent	모든 노출 숫자가 Tool output 또는 user input 에서 왔음을 audit 로 증명. unsupported number 생성 0 건
UI	Data Connections → Twin → Chat → Scenario → Evidence Drawer 가 하나의 데모 플로우로 연결
Portfolio	README 에 실제/테스트/Mock 경계, 아키텍처, 7 억 주택 데모, 제한사항, 검증 결과를 명시

최종 데모는 인터넷/외부 API 일시 장애가 있어도 재현 가능해야 한다. 따라서 동일 Open Banking response fixture 를 replay 하는 Demo Mode 를 함께 제공하되, Live Test Mode 와 화면에서 명확히 구분한다.

13. 테스트와 평가 기준

금융 AI 프로젝트의 품질은 "답변이 그럴듯한가"가 아니라 "숫자가 맞고 추적 가능한가"로 본다.

13.1 Deterministic Engine

- 상환스케줄: 공개된 금융계산기나 독립 스프레드시트와 golden case 비교.
- Snapshot: 동일 원천데이터에서 동일 Financial Twin 이 생성되는지 검증.
- Stress Test: shock 조합이 Base 데이터에 정확히 적용되는지 검증.
- Scenario Comparison: 서로 다른 시나리오가 동일 metric 정의를 사용하는지 검증.

13.2 Agent

평가항목	측정 방법	목표 예시
Tool Selection Accuracy	의도별 기대 Tool sequence 와 비교	대표 질문셋에서 높은 일치율
Unsupported Number Rate	Tool output 에 없는 수치 생성 여부	0%를 목표
Trace Completeness	주요 수치가 snapshot/assumption/rule 에 연결되는지	핵심 KPI 100%
Failure Handling	데이터 부족 시 추정 대신 입력요청/분석불가 처리	정의된 failure case 전부 통과
Scenario Reproducibility	scenarioId 로 동일 결과 재생	동일 버전 100%

13.3 데이터 품질

기존 LoanOps 의 Quality Validation 과 Quarantine 을 개인 금융 데이터에도 적용한다. 누락 거래, 중복 거래, 비정상 음수 잔액, 대출 잔액/납입 불일치 등 개인 데이터용 rule set 을 별도로 두어 Financial Twin 에 오염된 값이 들어가지 않게 한다. 비기능 지표로는 non-LLM simulation latency, Agent tool-call latency, 사용자 노출 수치의 audit coverage, product/personal snapshot freshness 를 함께 기록한다.

13.4 Product Matching

- Hard Filter correctness: 금액/기간/상환방식 조건을 위반한 상품이 후보에 남지 않는지 golden cases 로 검증.
- Ranking reproducibility: 동일 snapshot/publication/ruleVersion 에서 Top-K 와 score breakdown 이 동일한지 검증.
- Simulation consistency: 후보 A/B/C 모두 동일한 loan amount, term assumptions, Financial Twin metric 정의를 사용하도록 검증.
- Missing-data safety: eligibility/amount/rate 정보가 부족할 때 INSUFFICIENT_DATA 또는 warning 으로 처리하고 추정하지 않는지 검증.
- Freshness/quality: stale 또는 non-serving 상품이 ranking 상단에 노출되지 않는지 검증.

14. 대표 Demo Scenario

프로젝트 설명은 “7 억원 주택구매” 하나를 완벽하게 보여주는 것으로 충분하다.

14.1 Integrated Test Persona - 각 숫자의 출처를 명시

	예시 값	데이터 출처 / 생성 방식
	66,000,000 KRW	Mock Public MyData - 소득금액 eviden
자산	120,000,000 KRW	Open Banking Test - account balance
용대출	30,000,000 KRW	Open Banking Test - loan/lease basic
생활비	2,800,000 KRW	Open Banking Test 거래내역에서 6 개월
카드 청구	1,470,000 KRW	Open Banking Test - card billing (capability available 시)
택가격	700,000,000 KRW	사용자 입력

14.2 Demo Flow

1. 사용자가 “7 억 아파트를 사도 될까?”라고 입력한다.
2. Agent 가 Financial Twin 과 기존 부채를 조회한다.
3. 주택가격과 자기자본에 따라 필요한 주담대 시나리오를 만든다.
4. Product Matching Engine 이 LoanOps Serving 데이터에서 목적·금액·기간 조건이 맞는 후보 풀을 만들고 Hard Filter 를 적용한다.
5. Top-K 후보 각각을 Home/Loan Simulator 에 적용해 월 상환액, 총이자, 구매 후 Balance Sheet 와 Cashflow 를 계산한다.
6. 각 후보에 동일한 금리 +1/+2%p, 소득 -20%, 복합 shock 을 적용해 Stress Resilience 를 계산한다.
7. Deterministic score 로 검토 후보를 정렬하고, Agent 가 후보별 trade-off 와 추가 확인이 필요한 자격조건을 설명한다.
8. 사용자가 원하면 가격하향/기존대출 상환/구매연기 대안까지 실행하고, 최종 화면에서 상품 후보·재무 변화·리스크·근거를 함께 보여준다.

14.3 결과 화면에서 보여줄 메시지 구조

```
1) 현재 상태
   - Net Worth / Free Cashflow / Debt / Emergency Fund

2) 구매 후 변화
   - 월 여유현금 감소
   - 유동자산 감소
   - 부채 증가

3) Stress Test
   - 금리 상승 시 월 현금흐름
   - 소득 감소 시 적자 전환 여부

4) Alternatives
   - 주택가격 낮추기
   - 기존대출 선상환
   - 구매 연기

5) Evidence
   - 상품 데이터 출처 / 가정 / 계산공식 / Snapshot ID
```

데모의 핵심

"살 수 있나?"에 답하는 대신 "사면 내 생활이 어떻게 바뀌며, 어떤 충격까지 버틸 수 있나?"를 보여준다.

15. 현실적인 제약과 범위 관리

신뢰도를 높이려면 무엇을 구현하지 않을지도 명확해야 한다.

제약	MVP 대응
Open Banking 실사용 데이터	개발자 사이트는 API 개발/테스트에 활용하고, 실제 고객 데이터 운영은 별도 이용계약·승인을 전제로 한다. Test persona 와 API contract 를 중심으로 데모한다.
Open Banking 테스트 데이터	개발자 환경에서 등록/설정한 테스트 응답과 sandbox 데이터를 사용한다. 이를 실제 고객 데이터라고 표현하지 않는다.
공공 MyData 직접 연동	이용기관 신청, 환경조사, 묶음정보 신청, 현장실사·심사 등이 필요하므로 개인 포트폴리오에서는 공식 계약에 맞춘 Mock Adapter 로 구현한다.
AI-Hub 합성데이터 결합	다른 출처의 레코드를 동일 개인이라고 억지로 join 하지 않는다. 대규모 offline 품질/분석/ML 실험과 fixture 생성에만 사용한다.
은행별 실제 승인모델 비공개	Estimated/Simulated 값으로 제공하고 "승인 가능"을 단정하지 않는다.
규칙 변경	DSR/LTV/세금 등은 Versioned Rule Layer 와 as-of date 로 관리한다.
개인정보 민감성	테스트 ID 사용, token 암호화, 로그 redaction, 최소수집/동의 scope, raw sensitive evidence 보관 최소화.

15.1 MVP 에서 과감히 제외할 것

- 신용평가 모델 자체 개발
- 실제 은행 대출신청 자동 실행
- 복잡한 투자 포트폴리오 최적화
- 과도한 ML 모델(LSTM/Transformer)을 데이터 근거 없이 추가
- microservice/Kafka 등 기능 가치와 직접 관련 없는 인프라 과설계

15.2 이후 확장

- Cashflow Forecasting 모델 추가
- Monte Carlo 기반 장기 자산/부채 시뮬레이션
- 대출 상환 vs 투자 등 자산배분 의사결정
- 정책금융/지원금 데이터 연결
- Open Banking 정식 이용계약 및 공공 MyData 이용기관 승인 후 실제 사용자 데이터 Provider 로 교체
- 사용자 동의 기반 반자동/자동 금융 Action

16. 포트폴리오 스토리

"AI 를 붙였다"가 아니라 "기존 데이터 인프라가 있었기 때문에 신뢰할 수 있는 AI 의사결정 서비스를 만들었다."

16.1 프로젝트 발전 서사

단계	설명
LoanOps v1	여러 금융 API 의 데이터를 수집·정규화하고, 품질검증·오류격리·변경이력 관리로 신뢰 가능한 금융상품 데이터 구축
문제 발견	좋은 상품데이터만으로는 사용자의 실제 금융 의사결정을 설명하기 어렵다는 한계
LoanOps AI	개인 Financial Twin 과 상품데이터를 결합하고 대출/주택구매의 미래 영향을 Simulation
Agentic UX	자연어 목표를 Tool 로 변환하고, 시나리오를 비교한 뒤 근거와 함께 설명

16.2 이 프로젝트에서 드러나는 역량

역량	증거
FinTech Domain	대출·상환·금리·유동성·스트레스 시나리오 모델링
Data Engineering	Raw → Normalize → Quality → Serving 파이프라인
Backend Engineering	도메인모델, API, rule engine, audit
Data Analytics	Financial Twin, cashflow/debt/liquidity metrics
AI / Agent	Tool calling, scenario orchestration, traceable explanation
AX	조회 중심 금융서비스를 분석·시뮬레이션·선제 알림 중심으로 전환

16.3 README/면접에서 사용할 한 문장

Portfolio Pitch
"기존 LoanOps 의 신뢰 가능한 금융상품 파이프라인에 금융결제원 Open Banking 개발/테스트 API 와 공공 MyData 계약형 Adapter 를 연결해 Personal Financial Twin 을 구축했습니다. 사용자의 대출 목적과 재무여력을 먼저 계산한 뒤 LoanOps 의 검증된 상품에서 후보를 필터링하고, 각 후보를 동일한 월 상환·현금흐름·금리 Stress 시뮬레이션에 넣어 비교하는 Product Matching Engine 을 추가했습니다. AI Agent 는 이 deterministic Tool 결과와 데이터 출처를 추적하며 후보별 trade-off 를 설명하도록 설계했습니다."

17. 참고자료 및 정책 배경

프로젝트 방향성을 설명하기 위한 최신 공공자료. 구현 시 규칙/규제는 별도 최신 검증 필요.

금융위원회, 2026-08-25 - “AI 시대 국민의 일상과 생업을 잇는 마이데이터 발전방안을 추진합니다.” - 개인사업자 마이데이터, My AI 에이전트 도입, 데이터 활용 확대 논의. [원문]

금융위원회, 2026-06-17 - “마이데이터 활용 금리인하요구 서비스” 등 혁신금융서비스 신규 지정. [원문]

금융위원회, 2026-06-18 - 정부와 금융권이 함께 금융권 AX 를 가속화하고 금융분야 인공지능 가이드라인을 공개. [원문]

금융위원회, 2024-04-04 - “국민의 자산관리에 실질적 도움이 될 수 있는 마이데이터 2.0 을 추진하겠습니다.” [원문]

금융결제원 Open Banking 서비스 소개 - 잔액/거래내역/카드/보험/대출·리스 API 범위: https://openapi.kftc.or.kr/service/openBanking

금융결제원 개발자사이트 회원가입 - 금융 Open API 에 관심 있는 누구나 개발자사이트 가입 가능: https://developers.kftc.or.kr/dev/user/register

금융결제원 개발자 시작하기 - 서비스 신청, API Key, 테스트 정보 등록, 토큰 및 REST API 테스트: https://developers.kftc.or.kr/dev/starter/starter

금융결제원 OAuth 2.0 가이드: https://developers.kftc.or.kr/dev/openapi/open-banking/oauth

금융결제원 거래내역조회 API - 페이지당 최대 25 건: https://developers.kftc.or.kr/dev/openapi/open-banking/transaction

행정안전부 공공 MyData 이용기관 신청/승인 절차: https://adm.mydata.go.kr/intro/addHelpPage.do?type=index

공공 MyData 서비스 수행 가이드 - 본인정보 요청/수취 API 및 연계 테스트: https://adm.mydata.go.kr/images/guide.pdf

AI-Hub 금융 합성 데이터 - 카드/CB/금융상품 등 금융 합성 데이터: https://www.aihub.or.kr/aihubdata/data/view.do?aihubDataSe=data&dataSetSn=71792

문서 사용 시 주의

본 문서의 수치 예시는 기능 설명을 위한 synthetic example 이다. 실제 서비스 구현 시 대출 규제, 상품 조건, 금리, 세금, 부대비용, 개인정보 처리 요건은 적용 시점의 공식 자료와 전문가 검토를 통해 별도 검증해야 한다.

최종 MVP 권고
Open Banking Test Integration + Public MyData Mock Adapter + Personal Financial Twin + Home Purchase Simulator + Loan Product Matching Engine + Stress Test + AI Agent. 핵심 Demo 는 “7 억원 주택구매”에서 필요한 대출규모를 계산하고, LoanOps 상품 Top-K 를 후보별로 재시뮬레이션하여 사용자에게 근거와 trade-off 를 설명하는 흐름으로 완성한다.